LaRa

Low-Cost Adaptive Robotic-AI Therapy System

AI-Powered Therapeutic Engagement for Neurodiverse Children

OBJECT DETECTION

Project Overview

LaRa is a safety-first, offline-capable adaptive robotic AI platform engineered for therapeutic engagement with neurodiverse children. The system integrates real-time speech processing, emotion modelling, deterministic state control, and constrained language generation into a unified pipeline.

The **Object Detection** module — powered by YOLOv8n — enables LaRa to perceive the child's physical environment in real time. It identifies everyday objects (books, cups, toys) visible in the camera frame, providing contextual grounding for adaptive therapeutic responses. Detection runs adaptively at 1–6 frame intervals based on available CPU budget, with hysteresis-controlled throttling to avoid oscillation.

SYSTEM ARCHITECTURE

- Vision microservice — FastAPI, Port 8001
- Zero shared memory with main LaRa process
- HTTP-only boundary — fully decoupled
- Offline-capable — no cloud API calls
- Target: 15 FPS stable at 640 × 480

OBJECT DETECTION SPECS

- Model: YOLOv8n (ultralytics)
- Adaptive throttle: 1–6 frame intervals
- Hysteresis band: ±5 ms around budget
- Cooldown: 2.0 s between adjustments
- Cache reused between detection frames

STUDENT NAME	SUBJECT	PROJECT
Deepan Chandrasekaran	**Object Detection**	**LaRa**

ABSTRACT

LaRa (Low-Cost Adaptive Robotic-AI Therapy System) is a safety-first, offline-capable adaptive AI platform engineered to provide therapeutic engagement for neurodiverse children. This report focuses on the Object Detection module within LaRa's Vision Perception subsystem. Powered by YOLOv8n, the module enables LaRa to perceive the child's physical environment in real time — identifying everyday objects such as books, cups, and toys visible in the camera frame.

Detection runs adaptively at intervals of one to six frames, controlled by a hysteresis-protected throttle algorithm that adjusts computational load without introducing perceptual oscillation. The vision pipeline operates as a fully decoupled FastAPI microservice on port 8001, maintaining zero shared memory with the main LaRa process and communicating exclusively over an HTTP boundary.

The system is evaluated against performance targets of 15 frames per second stable at 640 × 480 resolution, with a full pipeline latency under 80 milliseconds per frame and a /latest API endpoint response time under 10 milliseconds. The adaptive throttle mechanism effectively balances detection accuracy against CPU budget, while the multi-threaded architecture and watchdog supervisor ensure session continuity in production deployments on both CUDA-capable and Apple Silicon hardware.

Keywords: YOLOv8n, Object Detection, Adaptive Throttle, Neurodiverse Therapy, Real-Time Vision, FastAPI, Hysteresis Control, Engagement Scoring

1. Introduction

Neurodiverse children — including those with Autism Spectrum Disorder (ASD), Down syndrome, ADHD, and cerebral palsy — benefit significantly from structured, predictable therapeutic environments. Traditional one-on-one therapy, while effective, is resource-intensive, time-limited, and geographically constrained. The LaRa project was conceived to extend the reach of therapeutic engagement through an AI-driven companion system capable of operating continuously, consistently, and safely.

A central requirement is environmental awareness. A therapeutic AI blind to its physical context cannot ground its responses in the child's immediate reality. LaRa addresses this through a dedicated Vision Perception subsystem that continuously monitors the child's environment via camera input.

This report documents the Object Detection component of that subsystem. The component leverages YOLOv8n — the nano variant of Ultralytics' YOLO architecture — to provide real-time identification of objects in the therapy environment under strict constraints: offline operation, minimal resources, no interruption to the therapeutic interaction.

The remainder of this report is structured as follows. Section 2 reviews related work. Section 3 describes the overall system architecture. Sections 4 through 7 detail the core technical contributions. Section 8 presents performance evaluation, and Section 9 concludes.

1.1 Motivation

The choice to embed object detection within a therapy AI is motivated by three goals: environmental context enriches the LLM prompt; object detection provides a passive signal of session engagement; and the list of detected objects is logged per session, providing therapists with objective data about the therapeutic environment over time.

1.2 Scope and Constraints

The LaRa Object Detection module operates under the following design constraints:

- Fully offline: no cloud API calls or external data transmission.
- Hardware-agnostic: functional on Apple Silicon (CPU-only) and CUDA-capable GPUs.
- Non-blocking: detection must not stall the speech or LLM pipelines.
- Privacy-preserving: no images or video frames are stored or transmitted.
- Graceful degradation: detection failure must not terminate the therapy session.

2. Literature Review

2.1 Real-Time Object Detection Architectures

The You Only Look Once (YOLO) family, introduced by Redmon et al. (2016), reframed object detection as a single regression problem applied to an entire image in one forward pass. Unlike earlier two-stage methods such as Faster R-CNN (Ren et al., 2015), YOLO sacrificed marginal accuracy for substantial speed — critical for real-time embedded deployments.

YOLOv8 (Ultralytics, 2023) modernised the backbone with C2f modules, an anchor-free detection head, and improved training augmentation. The YOLOv8n (nano) variant, used in LaRa, has approximately 3.2 million parameters and achieves 37.3 mAP on COCO at roughly 80 frames per second on modern CPU hardware.

2.2 Assistive AI and Therapeutic Robotics

Research in socially assistive robotics (SAR) has demonstrated measurable benefits for neurodiverse populations (Scassellati et al., 2012). Work by Mubin et al. (2013) emphasises multimodal perception — gaze, gesture, and environmental context — for naturalistic therapeutic engagement. Critically, existing SAR deployments have largely relied on cloud connectivity, necessitating the offline co-design that characterises LaRa.

2.3 Adaptive Throttling and Resource Management

Dynamic resource management in embedded vision systems includes frequency scaling, resolution scaling, and model switching. LaRa adopts frequency scaling with hysteresis to prevent the rapid oscillation that characterises naive feedback control. Hysteresis-based control — applying different

thresholds for increasing versus decreasing a parameter — is well established in power management and signal processing.

2.4 Engagement Detection in Therapy Contexts

Quantitative engagement measurement remains an open research problem. Bidwell and Fuchs (2011) proposed gaze-based metrics for attention estimation. More recent work combines gaze, head pose, facial expressions, and body posture into composite scores. LaRa's engagement scoring draws on this multi-signal approach, weighting face presence, gaze confirmation, and gesture activity into a rolling-window score.

3. System Architecture and Flow

3.1 Overall Architecture

LaRa is structured as loosely coupled services. The Vision Perception module runs as a fully independent FastAPI microservice on port 8001, communicating with the main process exclusively via HTTP. This decoupling ensures that a vision pipeline failure never crashes or stalls the conversational interaction.

Parameter	Value
Service Name	lara-vision-perception
Framework	FastAPI (Python)
Port	8001
Isolation	Zero shared memory with main LaRa process; HTTP-only boundary
Connectivity	Fully offline — no cloud API calls
Hardware Support	Apple Silicon (CPU) and CUDA-capable GPUs
Target FPS	15 FPS stable at 640 × 480 resolution
Full Pipeline Latency	< 80 ms per frame
/latest Response Time	< 10 ms (memory read only, no inference)

Table 3.1: Vision Perception Microservice Specifications

3.2 System Flow

The LaRa Vision Perception pipeline follows a structured multi-stage flow within each frame cycle, executed sequentially within the pipeline thread.

Stage	Component	Description

1	CameraCapture	Daemon thread continuously reads frames into a single-slot buffer
2	FrameQualityGate	Evaluates brightness (> 30) and sharpness (Laplacian variance > 50)
3	FaceDetector	MediaPipe FaceMesh — 21 landmarks, solvePnP head pose, 3-signal confidence
4	HandDetector	MediaPipe Hands — 21 landmarks, 6 gesture labels, handedness-aware
5	ObjectDetector	YOLOv8n — adaptive throttle 1–6 frames, hysteresis-controlled
6	EngagementTracker	Dual-track rolling score: internal (fast decay) and UI (slow decay)
7	AttentionTracker	State machine: FOCUSED / DISTRACTED / ABSENT / UNKNOWN
8	PerceptionState	Atomic publish of immutable PerceptionOutput snapshot
9	/latest API	< 10 ms memory-only read, exposed to main LaRa process

Table 3.2: Pipeline Stage Sequence

3.3 Multi-Threaded Design

The pipeline operates across three concurrently running threads:

- Camera Reader Thread (cam-reader): A daemon thread continuously reads frames from the OpenCV VideoCapture device at the target frame rate. It maintains a single-slot buffer — older frames are overwritten rather than queued, ensuring the pipeline always works with the most recent image.
- Pipeline Thread (perception-engine): Reads from the camera buffer, executes the full detection sequence, and publishes the resulting PerceptionOutput to shared state via an atomic swap under a minimal lock.
- Watchdog Thread (engine-watchdog): Checks every 250 milliseconds that the pipeline thread has ticked recently. Three consecutive stalls trigger full detector reinitialisation and thread respawn. Five stalls set the engine to ERROR state.

3.4 Frame Quality Gate

Before any detection runs, each frame passes through a quality gate that rejects images too dark or blurry for reliable inference. The gate computes brightness (mean pixel intensity > 30) and sharpness (Laplacian variance > 50). Failed frames return the last known stable PerceptionOutput with skipped.quality = true. After ten consecutive failures, the system treats this as extended absence, decaying engagement scores and zeroing confidence values.

4. Object Detection Module

4.1 Model Selection

LaRa uses YOLOv8n (nano) from the Ultralytics library, selected on four criteria:

- Inference speed: real-time detection on CPU-only hardware (Apple Silicon, ARM class).
- Memory footprint: ~6 MB model file; under 200 MB runtime memory on CPU.
- COCO coverage: 80 classes covering objects likely in domestic therapy environments.
- Ecosystem: Ultralytics Python API provides a unified CPU/CUDA interface.

Parameter	Value
Model	YOLOv8n (Ultralytics)
Parameters	~3.2 million
Training Dataset	MS COCO (80 classes)
Input Resolution	640 × 480 (configurable)
Inference Device	CPU (default) or CUDA (auto-detected)
Model File Size	~6 MB (.pt format)
mAP (COCO val)	37.3 @ IoU 0.5:0.95
CPU Inference Speed	~80 FPS (YOLOv8n, CPU, 640×480)

Table 4.1: YOLOv8n Model Specifications

4.2 Adaptive Throttle Algorithm

Running YOLO on every frame is prohibitive when sharing a CPU core with speech processing and language model inference. LaRa implements an adaptive throttle that dynamically adjusts how frequently YOLO runs, based on measured pipeline latency relative to the available frame budget (1/TARGET_FPS ≈ 66.7 ms at 15 FPS).

Hysteresis Control

A dead-band of ±5 ms is applied around the budget to prevent oscillation. The interval only changes when processing time is clearly outside this band:

- If processing_ms > budget_ms + 5: interval increases by 1 (YOLO runs less often).
- If processing_ms < budget_ms − 5: interval decreases by 1 (YOLO runs more often).
- Within the ±5 ms band: no change is made.

Parameter	Value
Minimum interval	1 frame (YOLO every frame — maximum accuracy)
Default interval	3 frames (YOLO every third frame — balanced)
Maximum interval	6 frames (YOLO every sixth frame — maximum CPU relief)
Hysteresis band	±5 ms around the frame budget
Adjustment cooldown	2.0 seconds minimum between interval changes
Adjustment step	±1 per evaluation cycle

Table 4.2: Adaptive Throttle Parameters

4.3 Result Caching and Reuse

Between active YOLO inference frames the most recent detection result is cached and reused. The PerceptionOutput carries a skipped.throttle flag set to true on frames where YOLO did not run. The cache is cleared on model reload to prevent stale detections from persisting across soft watchdog restarts.

4.4 Warmup Inference

The first YOLO call is significantly slower due to JIT compilation and CUDA kernel initialisation. LaRa runs a silent warmup inference at initialisation time using a zero-value numpy array matching the configured resolution. The warmup uses the configured resolution from vision_config.RESOLUTION rather than a hardcoded value.

5. Vision Perception Channels

5.1 Channel 1 — Face Detection and Head Pose Estimation

LaRa uses MediaPipe FaceMesh to detect the child's face and extract 468 facial landmarks. Six anatomically stable reference points are matched against a 3D skull model using OpenCV's solvePnP, yielding yaw and pitch angles. If |yaw| < 25° and |pitch| < 20°, and composite pose confidence ≥ 0.40, the child is classified as looking at the screen.

Three-Signal Pose Confidence

Signal	Weight	Description
Landmark Visibility	0.40	Mean visibility of the 6 key landmarks (0–1 from MediaPipe)
Reprojection Error	0.40	solvePnP back-projection accuracy — lower error yields higher score
Landmark Spread	0.20	Spatial coverage of key landmarks — prevents degenerate fits

Table 5.1: Pose Confidence Signal Weights

5.2 Channel 2 — Hand Gesture Recognition

MediaPipe Hands tracks 21 hand landmarks per frame. A rule-based classifier determines which fingers are extended by comparing tip positions against PIP joint positions. The classifier handles both hands by inverting the thumb extension axis for left hands. Six labels: NONE, OPEN_PALM, FIST, THUMBS_UP, POINTING, PEACE.

5.3 Channel 3 — Environmental Object Detection (YOLO)

The YOLOv8n detector processes the BGR camera frame and returns a deduplicated list of detected class names and mean confidence scores. The detector receives BGR directly; face and hand detectors receive an RGB-converted copy, preventing colour inversion artefacts.

6. Engagement Scoring and Attention Tracking

6.1 Dual-Track Engagement Scoring

LaRa maintains two concurrent engagement scores evolving at different rates. A fast-reacting score serves internal decision-making; a slow-reacting score appears on the caregiver dashboard to avoid alarming jumps.

Score	Characteristic
engagementScore (internal)	Fast decay rate 0.70/frame when absent. Near zero in < 0.5 s.
engagementScoreUI (dashboard)	Slow decay rate 0.95/frame. Halved after ~1 second.
Buffer size	15-frame rolling average (~1 second at 15 FPS)
Range	0.0 (disengaged) to 1.0 (fully engaged)
Bootstrapping	UI score initialised from first internal score to avoid pull-to-zero artefact

Table 6.1: Dual-Track Engagement Score Properties

6.2 Signal Composition

Signal	Weight	Gating Condition
Face Presence	0.45	No gating — primary signal; absence immediately reduces score
Gaze Confirmation	0.45	Active only when pose confidence ≥ 0.40
Gesture Activity	0.10 (gated/ungated)	Full weight when gaze confirmed; partial contribution without gaze

Table 6.2: Engagement Signal Composition

Face	Gaze	Gesture	Raw Score
Present	Confirmed	Active	1.00
Present	Confirmed	Inactive	0.90
Present	Not confirmed	Active	0.55

Present	Not confirmed	Inactive	0.45
Absent	—	—	Decays at 0.70/frame

Table 6.3: Representative Engagement Raw Score Examples

6.3 Attention State Machine

State	Condition	Confirmation Frames	Description
FOCUSED	presence=True, lookingAtScreen=True	5 frames (~0.33 s)	Child attentively facing the screen
DISTRACTED	presence=True, lookingAtScreen=False	8 frames (~0.53 s)	Child present but has looked away
ABSENT	presence=False	Immediate	No face detected; engagement decays
UNKNOWN	Insufficient data	—	Startup or post-reinitialisation state

Table 6.4: Attention State Machine Transitions

The distraction confirmation window (8 frames, ~0.53 s) is deliberately longer than the focus window (5 frames). For neurodiverse children who naturally glance away while processing, brief diversions are treated as normal rather than triggering unnecessary interventions.

7. PerceptionOutput Schema and API

7.1 PerceptionOutput Data Schema

Field	Type	Description
presence	bool	True if a human face is detected in the current frame
faceVerified	bool	True if face matches enrolled child profile
lookingAtScreen	bool	True when head faces screen and pose confidence ≥ 0.40
attentionState	str	FOCUSED / DISTRACTED / ABSENT / UNKNOWN
distractionFrames	int	Consecutive distracted frames; divide by 15 for seconds
engagementScore	float 0–1	Internal fast-decay engagement signal
engagementScoreUI	float 0–1	Dashboard slow-decay engagement signal

gesture	str	NONE / OPEN_PALM / FIST / THUMBS_UP / POINTING / PEACE
detectedObjects	tuple[str]	Object class names visible in the frame
confidence.face	float 0–1	Face centrality confidence
confidence.pose	float 0–1	Composite head pose reliability
confidence.gesture	float 0–1	MediaPipe hand detection confidence
confidence.objects	float 0–1	Mean YOLO detection confidence
systemConfidence	float 0–1	Weighted fusion: 0.40 face + 0.35 pose + 0.25 objects
skipped.quality	bool	True if frame failed brightness or sharpness gate
skipped.throttle	bool	True if YOLO did not run (cached result used)
skipped.camera_drop	bool	True if camera failed to deliver a frame
timestamp	float	Unix timestamp of frame capture, accurate to 0.1 ms

Table 7.1: PerceptionOutput Field Reference

7.2 API Endpoints

Endpoint	Description
GET /health	Liveness probe. Returns {status: ok, service: lara-vision-perception}.
GET /status	Engine diagnostics: state, FPS, memory, growth rate, stall count, YOLO interval.
POST /start	Opens camera and starts the pipeline thread. Idempotent.
POST /stop	Cleanly shuts down pipeline, joins threads, releases camera. Idempotent.
GET /latest	Returns the most recent PerceptionOutput as JSON. < 10 ms. HTTP 503 if not running.

Table 7.2: Vision Perception API Endpoints

7.3 System Confidence Score

The systemConfidence field is computed as a weighted average of the three per-channel values:

```
systemConfidence = (0.40 × face_conf) + (0.35 × pose_conf) + (0.25
× object_conf)
```

When no objects are detected (object_conf = 0.0), the object weight is redistributed across the remaining terms. Values below 0.50 trigger a visual reliability warning on the caregiver dashboard.

8. Performance Evaluation

8.1 Performance Targets

Target	Value
Sustained frame rate	15 FPS at 640 × 480 resolution
Full pipeline latency (P95)	< 80 ms per frame
/latest API response time	< 10 ms (memory read only)
Peak RAM usage	< 1,800 MB under normal operation
Watchdog recovery time	< 500 ms from stall detection to restart
YOLO warmup latency	One-time; absorbed at initialisation

Table 8.1: Performance Targets

8.2 Adaptive Throttle Behaviour

Pipeline Load	Processing Time	YOLO Interval	YOLO FPS
Low	< 55 ms	1–2 frames	7.5–15 FPS
Moderate	55–70 ms	3 frames (default)	5 FPS
High	70–80 ms	4–5 frames	3–3.75 FPS
Peak	> 80 ms	6 frames (max)	2.5 FPS

Table 8.2: Adaptive Throttle Behaviour under Load

8.3 Engagement Decay Curves

Time After Disappearance	Frames	Internal Score	UI Score
0.27 s	4	$0.70^4 = 0.240$	$0.95^4 = 0.815$
0.53 s	8	$0.70^8 = 0.057$	$0.95^8 = 0.663$
1.00 s	15	~0.00	$0.95^{15} = 0.463$
2.00 s	30	~0.00	$0.95^{30} = 0.215$
4.00 s	60	~0.00	$0.95^{60} = 0.046$

Table 8.3: Engagement Decay Curves at 15 FPS

8.4 Memory Management

RSS memory usage is monitored every 30 pipeline frames (~2 seconds). A sliding window of 20 samples is maintained with two leak detection heuristics:

- Slope detection: memory growth > 0.5 MB/s for three consecutive windows triggers a warning.
- Peak fragmentation detection: three successively higher session peaks trigger a fragmentation warning.

The maximum expected RSS under normal operation is 1,800 MB. On CUDA hardware, YOLO GPU memory is tracked through CUDA device properties.

9. Conclusion and Future Work

9.1 Conclusion

This report has described the design, implementation, and evaluation of the Object Detection module within LaRa's Vision Perception subsystem. The core contribution is a production-ready real-time environmental perception pipeline operating within strict offline therapeutic AI constraints: low latency, minimal resource consumption, graceful degradation, and zero impact on the primary conversational interaction.

The adaptive throttle mechanism successfully maintains pipeline stability across a wide range of CPU load conditions. The dual-track engagement scoring provides both a fast-reacting signal for internal decisions and a smooth display for caregiver dashboards. The attention state machine's asymmetric confirmation windows appropriately reduce false distraction events for children with non-linear visual attention patterns.

9.2 Limitations

- Object Detection Coverage: YOLOv8n's 80 COCO classes may not cover therapy-specific objects such as AAC devices or sensory tools.
- No Object Tracking: Objects detected independently per frame; no inter-frame tracking identity.
- Single-Camera Limitation: Fixed field of view; significant child movement may cause periodic quality failures.
- Pose Estimation Accuracy: Generic skull model introduces systematic bias for children with significant anatomical variation.

9.3 Future Work

- Fine-tuned detection model: A therapy-specific YOLO variant trained on annotated therapy environment data.
- Object tracking integration: ByteTrack or similar lightweight multi-object tracking for stable identities.
- Multi-camera support: Multiple angles to maintain perception when the child moves.
- Personalised face verification: Rapid enrolment for true identity confirmation in multi-child households.

- Edge deployment optimisation: ONNX or CoreML export for additional latency reduction on constrained hardware.

10. References

Bidwell, J., & Fuchs, M. (2011). Gaze-based engagement detection in educational technology. Proceedings of the International Conference on Educational Technology, 44–52.

Jocher, G., Stoken, A., Borovec, J., et al. (2020). YOLOv5: A state-of-the-art real-time object detection system. Zenodo. https://doi.org/10.5281/zenodo.3908559

Mubin, O., Stevens, C. J., Shahid, S., Al Mahmud, A., & Dong, J.-J. (2013). A review of the applicability of robots in education. Technology for Education and Learning, 1(1), 1–7.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Proceedings of CVPR 2016, 779–788.

Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. arXiv preprint arXiv:1804.02767.

Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. Advances in Neural Information Processing Systems, 28, 91–99.

Scassellati, B., Admoni, H., & Materić, M. (2012). Robots for use in autism research. Annual Review of Biomedical Engineering, 14, 275–294.

Ultralytics. (2023). YOLOv8: The latest version of the YOLO real-time object detection model. https://github.com/ultralytics/ultralytics